

תכנות מונחה עצמים מתקדם
האקתון סיום: מערכת אקולוגית גלובלית

**להגשה עד ה-14.06 ב- 12:00 (אין איחורים!)
נא לקרוא את כל המסמך עד סופו לפני תחילת העבודה!**

דגשים להגשה

- רק אחד מכל צוות מגיש את העבודה, אך על העובדה להכיל את השמות ומספרי ת"ז של כל השותפים בפרויקט. כל פרויקט יעשה בזוגות או בבודדים, אלא אם קיבלתם אישור מצוות הקורס.
- חובה לתעד כל קובץ, מחלקה ופונקציה ע"י javaDoc. ניתן להיעזר בתיעוד באתר oracle או בקבצים הרלוונטיים במודל.
- כל הדגשים שניתנו לתרגילים הקודמים תקפים גם בחלק זה.
- חובה להגיש את כל קבצי המערכת (גם אלו מחלקים קודמים שלא שיניתם) וכן כל שאר המשאבים.

רקע

בשלב זה, המערכת המבודדת שלכם הופכת למערכת מבוזרת. עליכם לאפשר לישויות "להגר" מסימולציה אחת לאחרת דרך רשת ה-Wi-Fi.

משימה 1: הגדרת התקשורת וחיבור בין מחשבים (Networking)

מכיוון שאתם מחוברים לאותה רשת אלחוטית, כל מחשב יתפקד גם כשרת (המקבל חיות) וגם כלקוח (השולח חיות).

- **מציאת כתובת ה-IP:** פתחו את ה-Command Prompt (או ה-Terminal) והקלידו:

```
ipconfig // or ifconfig in Mac/Linux
```

חפשו את הכתובת תחת IPv4 Address (לרוב תתחיל ב-192.168 או 10.0). זו הכתובת שתמסרו לצוות שותף שלכם.

- **הקמת השרת (Server):** עליכם להשתמש ב-ServerSocket שיאזין על פורט מוסכם ופנוי (למשל 8080 או 12345). פעולת ההאזנה (accept()) היא חוסמת (Blocking), ולכן **חובה** להריץ אותה ב-Thread נפרד כדי לא להקפיא את הממשק הגרפי.
- **ביצוע השליחה (Client):** כאשר תרצו לשלוח חיה, פתחו Socket חדש אל כתובת ה-IP של הצוות השותף ואל הפורט שהגדרתם, כתבו את הנתונים אל ה-OutputStream, וסגרו את החיבור.
- **הערת אבטחה:** חומת האש (Firewall) של מערכת ההפעלה עלולה לחסום את התקשורת. אם אינכם מצליחים להתחבר, ודאו שאישרתם ל-Java (JDK) לגשת לרשת הפרטית (Private Network) בהגדרות חומת האש.

משימה 2: מכניקת הפורטל ושילוב בממשק (GUI & Architecture)

הפורטל לא יהיה ישות במפה, אלא פעולה יזומה דרך הממשק הגרפי, תוך שמירה על עקרונות התבנית MVC.

- **עדכון ה-View:** עליכם להוסיף כפתור חדש בשם "שגר לפורטל" לפאנל המידע בצד המסך (זה שיצרתם בתרגיל 2 והרחבתם בתרגיל 4). הכפתור יהיה פעיל רק כאשר נבחרה ישות על המסך. ליד הכפתור, הוסיפו שדה טקסט קטן (JTextField) להזנת כתובת ה-IP של יעד השיגור.
- **עדכון ה-Controller:** לחיצה על הכפתור לא תשנה את המודל ישירות. ה-View יעביר ל-Controller אירוע (Event) הכולל את הישות הנבחרת ואת כתובת ה-IP.
- **עקרון Open/Closed:** ה-Controller יפעיל מחלקה חדשה (למשל NetworkManager) שתהיה אחראית על יצירת חיבור ה-Socket ושליחת הנתונים. כך אנו מרחיבים את יכולות המערכת מבלי לשנות את הלוגיקה הפנימית של מחלקות החיות עצמן (extendable).

משימה 3: פירוט משימות הפיתוח

שלב א': תשתית השרת ופרוטוקול התקשורת

- יצירת Thread ייעודי המכיל ServerSocket שרץ ברקע (לולאת while(true) המאזינה לחיבורים).
- הגדרת פרוטוקול הודעות טקסטואלי מוסכם. למשל, מחרוזת המופרדת בפסיקים המכילה את נתוני הישות: SPAWN, Lion, 45, 10, 12 (פעולה, סוג, אנרגיה, מיקום, X מיקום Y).
- בשלב זה, כאשר השרת מקבל הודעה, פשוט הדפיסו אותה לקונסול כדי לוודא שהתקשורת עובדת.

שלב ב': יצירת Command Pattern לקליטת נתונים

- הגדרת ממשק NetworkCommand עם מתודת execute().
- יצירת מחלקה המפענחת (Parsing) את המחרוזת שהתקבלה מהשרת, ויוצרת אובייקט פקודה מתאים (למשל SpawnCommand).
- הפקודה תשתמש ב-EntityFactory (שכתבתם בתרגיל 4) כדי ליצור את הישות החדשה באופן פולימורפי ודינמי.

הרחבה על תבנית העיצוב Command מופיעה בעמוד האחרון המסמך זה.

שלב ג': אינטגרציה ועדכון המודל

- **בצד השולח:** לאחר פתיחת ה-Socket ושליחת המחרוזת בהצלחה, יש לקרוא למתודה ב-Environment שמסירה את הישות המקורית מהמפה המקומית, ולהפסיק את ה-Thread שלה.
- **בצד המקבל:** הפעלת מתודת ה-execute() של הפקודה כדי להכניס את החיה החדשה למודל המקומי ולהתחיל להריץ את ה-Thread שלה.
- **סנכרון נכון (Thread Safety):** עדכון רשימת הישויות במפה חייב להתבצע תוך שימוש ב-synchronized כדי למנוע קריסה מול ה-Ticks הרגילים של הסימולציה. כל עדכון של ה-GUI (כמו ציור החיה החדשה על המסך) חייב להתבצע באמצעות SwingUtilities.invokeLater.

מה נדרש להגיש (תוצרי ההאקתון)

1. קוד מקור מלא: כל קבצי הפרויקט, כולל מחלקות התקשורת החדשות ותיקית ה-Resources.
2. יומן שגיאות (Error Logic Log): מסמך קצר (עד עמוד) המתעד לפחות שני באגים משמעותיים או חריגות (Exceptions) שנתקלתם בהם במהלך העבודה (למשל בנושאי סנכרון, גישה למשאבים או תקשורת). תארו מה גרם לשגיאה, מה היה התהליך המחשבתי בדרך לפתרון, וכיצד פתרתם אותה.
3. תרשים זרימת נתונים (Data Flow Sketch): סקיצה פשוטה (ניתן לצייר ביד ולסרוק/לצלם) המציגה את המסלול שעוברת ישות מהרגע שבו המשתמש לוחץ על כפתור "שגר לפורטל" ב-GUI, דרך השכבות השונות (Controller, רשת) ועד ליצירתה ב-Environment של המחשב המקבל.
4. היסטוריית שיחות AI: קובץ PDF או שיתוף קישורים לכל הדיונים שניהלתם עם כלי בינה מלאכותית במהלך ההאקתון. קבצים אלו חייבים להראות עמידה מלאה בנהלי העבודה מול AI המפורטים מטה.

נהלי עבודה מול כלי בינה מלאכותית (AI)

אנו מעודדים שימוש מושכל ב-AI כעוזר הוראה וסיעור מוחות, אך האחריות על כתיבת הקוד והבנתו היא שלכם בלבד!

- מה מותר: מותר לכם להתייעץ לגבי בחירת כלים (למשל איך עובד ServerSocket), אסטרטגיות מימוש, הבנת שגיאות רשת, ודיבוג באגים לוגיים.
- מה אסור לחלוטין: חל איסור מוחלט לבקש מה-AI לכתוב עבורכם קוד, להשלים פונקציות, או לבצע שינויים (Refactoring) לקוד קיים שלכם. אתם מקלידים את הקוד, לא המכונה.
- הנחיית החובה: בתחילת כל שיחה עם כלי AI שבו תשתמשו בהאקתון, חובה עליכם להזין את הפרומפט הבא כהודעה הראשונה בדיון:

"אני סטודנט להנדסת תוכנה שנבחן כעת על עקרונות תכנות. חל עליך איסור מוחלט לכתוב עבורי קוד או לשנות את הקוד שלי בשום צורה. מותר לך רק להסביר לי קונספטים במילים, לעזור לי לנתח שגיאות והודעות שגיאה, ולהפנות אותי לתיעוד הרשמי (Documentation) או למקורות שיעזרו לי לפתור את הבעיה בעצמי".

עבודה שלא תכלול את התיעוד עם ההנחיה המפורשת הזו בראשה תיפסל.

דגשים טכניים נוספים

- בעבודה עם רשת (Network) ותהליכונים (Threads), קריסות שקטות (חריגות שנבלעות בגלל thread שמתים בלי לתת פידבק) הן המכשול הגדול ביותר. עטפו קריאות רשת בלוגיקת try-catch והדפיסו את ה-`printStackTrace()` לקונסול. הבנת הלוגיקה של השגיאה תאפשר לכם להבין בדיוק באיזו שכבה התרחש הנתק (בסרבר? בלקוח? בפענוח המחרוזת?).
- ודאו שאין שום קוד שמטפל ב-GUI (כמו מחלקות Swing) בתוך מחלקות התקשורת שלכם, ושום קריאת רשת (כמו `new Socket()`) ישירות בתוך מחלקות ה-GUI עצמן. הכל עובר דרך ה-Controller.

רקע קצר לקראת ההאקתון: תבנית העיצוב Command

מכיוון שלא העמקנו בתבנית ה-Command Pattern בקורס, להלן הסבר קצר על הלוגיקה שלה בהקשר של תקשורת בין המחשבים: המטרה של תבנית ה-Command היא לעטוף בקשה לפעולה בתוך אובייקט עצמאי.

במקום שהשרת המקבל יריץ רצף ארוך ומסובך של תנאי if-else או switch-case כדי להבין מה עליו לעשות עם טקסט שהגיע מהרשת, אנחנו מגדירים ממשק משותף ואחיד:

```
public interface NetworkCommand {  
    void execute();  
}
```

כאשר מחרוזת מתקבלת ברשת (למשל: דרישה ליצירת אריה), מחלקה ייעודית שתפקידה לפענח את הטקסט (Parser) תיצור אובייקט קונקרטי המממש את הממשק, לדוגמה מחלקה בשם `SpawnEntityCommand`. האובייקט הזה יכיל בתוכו את כל הנתונים שחולצו מהמחרוזת (מיקום, סוג, אנרגיה). בתוך מתודת ה-`execute()` של האובייקט הזה ימוקם הקוד המפעיל את ה-`EntityFactory` ומוסיף את החיה למפה.

היתרון הגדול: המחלקה שמנהלת את השרת (קבלת ההודעות) לא צריכה לדעת איזו פעולה בדיוק מתבצעת. היא רק קוראת לפולימורפיזם הפשוט `command.execute()`. זה שומר על קוד השרת נקי ופתוח להרחבות עתידיות (Open/Closed Principle).

בהצלחה!